

Resúmenes

Modulo 3: Fundamentos de Python

• Comparación y condiciones

Los operadores de comparación permiten evaluar valores y obtener True o False. Entre ellos están: $=$ para igualdad, $!=$ desigualdad, $>$ para mayor que, $>=$ para mayor o igual que, $<$ para menor que, $<=$ para menor o igual que.

Las condiciones permiten que el programa tome decisiones. La sentencia `if` ejecuta un bloque cuando la condición es verdadera. `if-else` permite elegir entre dos alternativas, mientras que `elif` permite comprobar varias condiciones. También es posible utilizar condiciones anidadas.

• Bucles con while

El bucle `while` repite las instrucciones mientras su condición se true. A diferencia de `if`, que ejecuta las instrucciones una sola vez, `while` puede repetir las varias veces. Es importante modificar los valores que intervienen en la condición para evitar un bucle infinito.

• Bucles con for

El bucle con `for` se utiliza cuando se necesita repetir instrucciones y recorrer valores o colecciones. La función `range` genera los valores que se utiliza la variable de control. Puede utilizarse con uno, dos o tres argumentos para definir el inicio, final y paso.

Las instrucciones `break` y `continue` permiten controlar el funcionamiento de los bucles. `break` termina el bucle inmediatamente y `continue` omite el resto de la iteración actual y continúa con la siguiente. Los bucles `while` y `for` también pueden usarlos.

• Logica de computadora.

Los operadores lógicos permiten construir condiciones mas complejas, `and` requiere que ambas condiciones sean verdaderas.

or es verdadero cuando al menos una condición lo es; not invierte el valor lógico.

Las expresiones lógicas también permiten trabajar con las leyes de De Morgan. El documento diferencia los valores lógicos de los bits individuales: los operadores lógicos consideran el valor completo, donde cero representa false y un valor diferente es true.

• Operadores bit a bit

Trabajan directamente con los bits individuales de los valores. También se presentan los desplazamientos binarios hacia la izquierda y hacia la derecha, que permite mover los bits de una posición a otra.

• Listas

Las listas permiten almacenar varios elementos y acceder a ellos mediante índices. Los índices comienzan desde 0 y también pueden utilizarse índices negativos. La función Len sirve para saber la cantidad de elementos. Los elementos pueden agregarse con append al final de la lista o con insert en una posición determinada. También se puede eliminar mediante del.

• Listas en acción

Las listas pueden procesarse mediante ciclos para buscar valores, calcular resultados y recorrer sus elementos. Se muestra cómo encontrar el elemento mayor de una lista y cómo localizar un elemento determinado.

• Lista dentro de listas

Una lista puede encontrarse dentro de otra. Esta estructura permite representar arreglos, como un tablero de ajedrez. Para acceder a un elemento se usa dos

índices, uno para identificar la fila y otra para la columna.

- Arreglos bidimensionales

Las listas dentro de otras permiten trabajar con información organizada en filas y columnas.

- Arreglos tridimensionales

Python permite aumentar la profundidad de las listas dentro de listas.

Exámenes

PROGRA - Documentos de Go

Edube Interactive - PE1 -- Mod

+

Pregúntale a Gemini

edube.org/quiz/python-essentials-1-esp-edube/pe1-module-3-quiz-7

Python INSTITUTE

PE1 -- Module 3 Quiz

Progreso (100%)

≡

Tu puntaje: 10/10

100%

Congratulations, you've passed the quiz!

ANÁLISIS DE LA SECCIÓN

PE1 -- Module 3 Quiz100%

Volver a presentar la prueba

Revisar prueba

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376.jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

PROGRA - Documentos de Go

Edube Interactive - FP1: Exame

+

Pregúntale a Gemini

edube.org/quiz/python-essentials-1-esp-edube/fp1-examen-modulo-3

Python INSTITUTE

FP1: Examen Módulo 3

Progreso (100%)

≡

Tu puntaje: 19/20

95%

Congratulations, you've passed the test!

ANÁLISIS DE LA SECCIÓN

PE1 -- Module 3 Test95%

Volver a presentar la prueba

Revisar prueba

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376.jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Laboratorios

Python INSTITUTE

3.1.1.4 LABORATORIO: Preguntas y respuestas

MODULE (7%)

SECTION (93%)

Tiempo Estimado

5 minutos

Nivel de Dificultad

Muy Fácil

Objetivos

- Familiarizarse con la función `input()`.
- Familiarizarse con los operadores de comparación en Python.

Escenario

Usando uno de los operadores de comparación en Python, escribe un programa simple de dos líneas que tome el parámetro `n` como entrada, que es un entero, e imprime `False` si `n` es menor que `100`, y `True` si `n` es mayor o igual que `100`.

No debes crear ningún bloque `if` (hablaremos de ellos muy pronto). Prueba tu código usando los datos que te proporcionamos.

Datos de Prueba

Ejemplo de entrada: 55

Resultado esperado: `False`

Ejemplo de entrada: 99

Resultado esperado: `False`

```
1 n = int(input("Ingresa un número entero: "))
2 print(n >= 100)
```

Console >_

Ingresa un número entero: 55
False
Ingresa un número entero: 99
False

Prev

Next

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376.jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Python INSTITUTE

3.1.1.10 LABORATORIO: Operadores de comparación y ejecución condicional

MODULE (15%)

SECTION (73%)

Entrada en forma de la palabra "espatifilio" grita involuntariamente a la consola la siguiente cadena: "¡Espatifilo es la mejor planta de todas!"

Escribe un programa que utilice el concepto de ejecución condicional, tome una cadena como entrada y que:

- Imprima el enunciado "Si, ¡El ESPATIFILIO! es la mejor planta de todos los tiempos!" en la pantalla si la cadena ingresada es "ESPATIFILIO".
- Imprima "No, ¡quiero un gran ESPATIFILIO!" si la cadena ingresada es "espatifilo".
- Imprima "¡ESPATIFILIO!, ¡No [entrada]!" de lo contrario. Nota: [entrada] es la cadena que se toma como entrada.

Prueba tu código con los datos que te proporcionamos. ¡Y hazte de un ESPATIFILIO también!

Datos de Prueba

Entrada de muestra: `espatifilo`

Resultado esperado: `No, ¡quiero un gran ESPATIFILIO!`

Entrada de ejemplo: `pelargonio`

Resultado esperado: `¡ESPATIFILIO!, ¡No pelargonio!`

```
1 planta = input("Ingresa el nombre de la planta: ")
2
3 if planta == "ESPATIFILIO":
4     print("Si, ¡El ESPATIFILIO! es la mejor planta de todos")
5 elif planta == "espatifilio":
6     print("No, ¡quiero un gran ESPATIFILIO!")
7 else:
8     print("¡ESPATIFILIO!, ¡No " + planta + "!")
```

Console >_

Ingresa el nombre de la planta: `espatifilo`
¡ESPATIFILIO!, ¡No espatifilo!

Prev

Next

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376.jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

PROGRA - Documentos de Go...Edube Interactive : 3.1.1.11 LAI...+Pregúntale a Gemini

edube.org/learn/python-essentials-1-esp-edube/laboratorio-fundamentos-de-la-sentencia-if-else-1

Python INSTITUTE3.1.1.11 LABORATORIO: Fundamentos de la sentencia if-else

MODULE (16%)SECTION (80%)

18% del ingreso menos 556 pesos y 2 centavos (esta fue la llamada *exención fiscal*).

- Si el ingreso era superior a esta cantidad, el impuesto era igual a 14,839 pesos y 2 centavos, más el 32% del excedente sobre 85,528 pesos.

Tu tarea es escribir una **calculadora de impuestos**.

- Debe aceptar un valor de punto flotante: el ingreso.
- A continuación, debe imprimir el impuesto calculado, redondeado a pesos totales. Hay una función llamada `round()` que hará el redondeo por ti, la encontrarás en el código de esqueleto del editor.

Nota: Este país feliz nunca devuelve dinero a sus ciudadanos. Si el impuesto calculado es menor que cero, solo significa que no hay impuesto (el impuesto es igual a cero). Ten esto en cuenta durante tus cálculos.

Observa el código en el editor: solo lee un valor de entrada y genera un resultado, por lo que debes completarlo con algunos cálculos inteligentes.

Prueba tu código con los datos que hemos proporcionado.

Datos de Prueba

Entrada de muestra: 10000

Resultado esperado: El impuesto es: 1244.0 pesos

Entrada de muestra: 100000

Resultado esperado: El impuesto es: 19470.0 pesos

```
1 income = float(input("Introduce el ingreso anual: "))
2
3 if income < 85528:
4     tax = income * 0.18 - 556.2
5     if tax < 0:
6         tax = 0.0
7 else:
8     surplus = income - 85528
9     tax = 14839.2 + surplus * 0.32
10
11 tax = round(tax, 0)
12 print("El impuesto es:", tax, "pesos")
```

Console >_

Introduce el ingreso anual: 10000
El impuesto es: 1244.0 pesos

PrevNext

LightbulbChat

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

PROGRA - Documentos de Go...Edube Interactive : 3.1.1.12 LAI...+Pregúntale a Gemini

edube.org/learn/python-essentials-1-esp-edube/laboratorio-fundamentos-de-la-sentencia-if-elif-else-1

Python INSTITUTE3.1.1.12 LABORATORIO: Fundamentos de la sentencia if-elif-else

MODULE (17%)SECTION (87%)

Desde la introducción del calendario Gregoriano (en 1582), se utiliza la siguiente regla para determinar el tipo de año:

- Si el número del año no es divisible entre cuatro, es un *año común*.
- De lo contrario, si el número del año no es divisible entre 100, es un *año bisiesto*.
- De lo contrario, si el número del año no es divisible entre 400, es un *año común*.
- De lo contrario, es un *año bisiesto*.

Observa el código en el editor: solo lee un número de año y debe completarse con las instrucciones que implementan la prueba que acabamos de describir.

El código debe mostrar uno de los dos mensajes posibles, que son `Año Bisiesto` o `Año Común`, según el valor ingresado.

Sería bueno verificar si el año ingresado cae en la era Gregoriana y emitir una advertencia de lo contrario: `No dentro del periodo del calendario Gregoriano`. Consejo: utiliza los operadores `!=` y `%`.

Prueba tu código con los datos que hemos proporcionado.

Datos de Prueba

Entrada de muestra: 2000

Resultado esperado: Año Bisiesto

Entrada de muestra: 2015

Resultado esperado: Año Común

```
1 year = int(input("Introduce un año:"))
2
3 if year < 1582:
4     print("No esta dentro del periodo del calendario Gregoriano")
5 elif year % 4 != 0:
6     print("Año Común")
7 elif year % 100 != 0:
8     print("Año Bisiesto")
9 elif year % 400 != 0:
10    print("Año Común")
11 else:
12    print("Año Bisiesto")
```

Console >_

Introduce un año:2000
Año Bisiesto

PrevNext

LightbulbChat

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Python

3.2.1.3 LABORATORIO: Lo esencial del bucle while - Adivina el número secreto

MODULO (24%)

SECTION (100%)

LABORATORIO

Tiempo Estimado

15 minutos

Nivel de Dificultad

Fácil

Objetivos

Familiarizar al estudiante con:

- Utilizar el bucle while.
- Reflejar situaciones de la vida real en código de computadora.

Escenario

Un mago junior ha elegido un número secreto. Lo ha escondido en una variable llamada `secret_number`. Quiere que todos los que ejecutan su programa jueguen el juego *Adivina el número secreto* y adivine qué número ha elegido para ellos. ¡Quieren no admitir el número quedarán atrapados en un bucle sin fin para siempre! Desafortunadamente, él no sabe cómo completar el código.

Tu tarea es ayudar al mago a completar el código en el editor de tal manera que el código:

- Pedirá al usuario que ingrese un número entero.
- Utilizará un bucle `while`.
- Comprobará si el número ingresado por el usuario es el mismo que el número escogido por el mago. Si el número elegido por el usuario es diferente al número secreto del mago, el usuario debería ver el mensaje: `"¡Ja, ja! ¡Estás atrapado en mi bucle!"` y se le solicitará que ingrese un número nuevamente. Si el número ingresado por el usuario coincide con el número escogido por el mago, el número debe imprimirse en la pantalla, y el mago debe decir las siguientes palabras: `"¡Bien hecho, muggle! Eres libre ahora"`.

¡El mago está contando contigo! No lo decepciones.

IMPORTANTE

Por cierto, observa la función `print()`. La forma en que lo hemos utilizado aquí se llama impresión multilinea. Puede utilizar *comillas triples* para imprimir cadenas en varias líneas para facilitar la lectura del texto o crear un diseño especial basado en texto. Experimenta con ello.

```
1 secret_number = 777
2
3 print()
4
5 """
6 | Bienvenido a mi juego, muggle!
7 | Introduce un número entero
8 | y adivina qué número he
9 | elegido para ti.
10 | Entonces,
11 | ¿Cuál es el número secreto?
12 |
13 """
14
15 number = int(input("Introduce un número entero: "))
16
17 while number != secret_number:
18     print("¡Ja, ja! ¡Estás atrapado en mi bucle!")
19     number = int(input("Introduce el número nuevamente: "))
20
21 print("¡Bien hecho, muggle! Eres libre ahora")
22
```

Console >_

```
"""
| Bienvenido a mi juego, muggle!
| Introduce un número entero
| y adivina qué número he
| elegido para ti.
| Entonces,
| ¿Cuál es el número secreto?
|
|
Introduce un número entero: 55
¡Ja, ja! ¡Estás atrapado en mi bucle!
Introduce el número nuevamente:
```

PrevNext

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Python

3.2.1.6 LABORATORIO: Fundamentos del bucle for: el conteo

MODULO (28%)

SECTION (35%)

Nivel de dificultad

Muy fácil

Objetivos

Familiarizar al estudiante con:

- Utilizar el bucle `for`.
- Reflejar situaciones de la vida real en código de computadora.

Escenario

¿Sabes lo que es Mississippi? Bueno, es el nombre de uno de los estados y ríos en los Estados Unidos. El río Mississippi tiene aproximadamente 2,340 millas de largo, lo que lo convierte en el segundo río más largo de los Estados Unidos (el más largo es el río Missouri). ¡Es tan largo que una sola gota de agua necesita 90 días para recorrer toda su longitud!

La palabra *Mississippi* también se usa para un propósito ligeramente diferente: para contar *mississippily* (*mississippimente*).

Si no estás familiarizado con la frase, estamos aquí para explicarte lo que significa: se utiliza para contar segundos.

La idea detrás de esto es que agregar la palabra *Mississippi* a un número al contar los segundos en voz alta hace que suene más cercano al reloj, y por lo tanto "un Mississippi, dos Mississippi, tres Mississippi" tomará aproximadamente unos tres segundos reales de tiempo. A menudo lo usan los niños que juegan al escondite para asegurarse de que el buscador haga un conteo honesto.

Tu tarea es muy simple aquí: escribe un programa que use un bucle `for` para "contar

```
1 import time
2
3 for i in range(1, 6):
4     print(i, "Mississippi")
5     time.sleep(1)
6
7 print("¡Listos o no, ahí voy!")
```

Console >_

```
1 Mississippi
2 Mississippi
3 Mississippi
4 Mississippi
5 Mississippi
¡Listos o no, ahí voy!
```

PrevNext

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Python INSTITUTE

3.2.1.9 LABORATORIO: La sentencia break - Atascado en un bucle

LABORATORIO

Tiempo Estimado

10 minutos

Nivel de Dificultad

Fácil

Objetivos

Familiarizar al estudiante con:

- Utilizar la instrucción `break` en los bucles.
- Reflejar situaciones de la vida real en código de computadora.

Escenario

La instrucción `break` se implementa para salir/terminar un bucle.

Diseña un programa que use un bucle `while` y le pida continuamente al usuario que ingrese una palabra a menos que ingrese "chupacabra" como la palabra de salida secreta, en cuyo caso el mensaje "¡Has dejado el bucle con éxito". Debe imprimirse en la pantalla y el bucle debe terminar.

No imprimas ninguna de las palabras ingresadas por el usuario. Utiliza el concepto de ejecución condicional y la sentencia `break`.

```
1 while True:
2     palabra = input("Introduce una palabra: ")
3     if palabra == "chupacabra":
4         break
5
6 print("¡Has dejado el bucle con éxito.")
```

Console >_

Introduce una palabra: jairo
Introduce una palabra:

Prev

Next

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Python INSTITUTE

3.2.1.10 LABORATORIO: La sentencia continue - El Feo Devorador de Vocales

El concepto de ejecución condicional (*if-elif-else*).

La sentencia `continue`.

Tu programa debe:

- Pedir al usuario que ingrese una palabra.
- Utiliza `user_word = user_word.upper()` para convertir la palabra ingresada por el usuario a mayúsculas; hablaremos sobre los llamados **métodos de cadena** y el `upper()` muy pronto, no te preocupes.
- Utiliza la ejecución condicional y la instrucción `continue` para "comer" las siguientes vocales *A, E, I, O, U* de la palabra ingresada.
- Imprime las letras no consumidas en la pantalla, cada una de ellas en una línea separada.

Prueba tu programa con los datos que le proporcionamos.

Datos de Prueba

Entrada de muestra: Gregory

Salida esperada:

G
R
G
R
Y

Entrada de muestra: abstemious

```
1 user_word = input("Ingresa una palabra: ")
2 user_word = user_word.upper()
3
4 for letter in user_word:
5     if letter == "A":
6         continue
7     elif letter == "E":
8         continue
9     elif letter == "I":
10        continue
11    elif letter == "O":
12        continue
13    elif letter == "U":
14        continue
15    else:
16        print(letter)
17
```

Console >_

Ingresa una palabra: Gregory
G
R
G
R
Y

Prev

Next

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Python INSTITUTE

« 3.2.1.11 LABORATORIO: La sentencia continue - El Bonito Devorador de Vocales »

MODULE (35%)

SECTION (65%)

la variable en la pantalla.

Analiza el código en el editor. Hemos creado `word_without_vowels` y le hemos asignado una cadena vacía. Utiliza la operación de concatenación para pedirle a Python que combine las letras seleccionadas en una cadena más larga durante los siguientes giros de bucle, y asignarlo a la variable `word_without_vowels`.

Prueba tu programa con los datos que le proporcionamos.

Datos de Prueba

Entrada de muestra: `Gregory`

Salida esperada:

`GRGRY`

Entrada de muestra: `abstemious`

Salida esperada:

`BSTMS`

Entrada de muestra: `IOUEA`

Salida esperada:

```
1 word_without_vowels = ""
2
3 user_word = input("Ingresa una palabra: ")
4 user_word = user_word.upper()
5
6 for letter in user_word:
7     if letter == "A":
8         continue
9     elif letter == "E":
10        continue
11    elif letter == "I":
12        continue
13    elif letter == "O":
14        continue
15    elif letter == "U":
16        continue
17    else:
18        word_without_vowels += letter
19
20 print(word_without_vowels)
```

Console >_

Ingresa una palabra: `Gregory`
`GRGRY`

Prev

Next

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Python INSTITUTE

« 3.2.1.14 LABORATORIO: Fundamentos del bucle while »

MODULE (39%)

SECTION (62%)

bloques.

Nota: La altura se mide por el número de **capas completas**: si los constructores no tienen la cantidad suficiente de bloques y no pueden completar la siguiente capa, terminan su trabajo inmediatamente.

Prueba tu código con los datos que hemos proporcionado.

Datos de Prueba

Entrada de muestra: `6`

Salida esperada: `La altura de la pirámide es: 3`

Entrada de muestra: `20`

Salida esperada: `La altura de la pirámide es: 5`

Entrada de muestra: `1000`

Salida esperada: `La altura de la pirámide es: 44`

Entrada de muestra: `2`

Salida esperada: `La altura de la pirámide es: 1`

```
1 blocks = int(input("Ingresa el número de bloques: "))
2
3 height = 0
4 in_layer = 1
5
6 while blocks >= in_layer:
7     height += 1
8     blocks -= in_layer
9     in_layer += 1
10
11 print("La altura de la pirámide es:", height)
```

Console >_

Ingresa el número de bloques: `6`
`La altura de la pirámide es: 3`

Prev

Next

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Python INSTITUTE

3.2.1.15 LABORATORIO: Hipótesis de Collatz

MODULE (90%)

SECTION (88%)

Escribe un programa que lea un número natural y ejecute los pasos anteriores siempre que `c0` sea diferente de 1. También queremos que cuente los pasos necesarios para lograr el objetivo. Tu código también debe mostrar todos los valores intermedios de `c0`.

Sugerencia: la parte más importante del problema es como transformar la idea de Collatz en un bucle `while` - esta es la clave del éxito.

Prueba tu código con los datos que hemos proporcionado.

Test Data

Entrada de muestra: 15

Salida esperada:

46
23
70
35
106
53
160
80
40
20
10
5
16
8
4
2
1
pasos = 17

Entrada de muestra: 16

Salida esperada:

8
4
2
1

```
1 c0 = int(input("Introduce un número entero positivo: "))
2
3 pasos = 0
4
5 while c0 != 1:
6     if c0 % 2 == 0:
7         c0 = c0 // 2
8     else:
9         c0 = 3 * c0 + 1
10    print(c0)
11    pasos += 1
12
13 print("pasos =", pasos)
```

Console >_

Introduce un número entero positivo: 15
46
23
70
35
106
53
160
80
40
20
10
5

PrevNext

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Python INSTITUTE

3.4.1.6 LABORATORIO: Lo básico de las listas

MODULE (59%)

SECTION (43%)

LABORATORIO

Tiempo Estimado

5 minutos

Nivel de Dificultad

Muy fácil

Objetivos

Familiarizar al estudiante con:

- Usar instrucciones básicas relacionadas con listas.
- Crear y modificar listas.

Escenario

Había una vez un sombrero. El sombrero no contenía conejo, sino una lista de cinco números: 1, 2, 3, 4 y 5.

Tu tarea es:

- Escribir una línea de código que solicite al usuario que reemplace el número central en la lista con un número entero ingresado por el usuario (Paso 1).
- Escribir una línea de código que elimine el último elemento de la lista (Paso 2).
- Escribir una línea de código que imprima la longitud de la lista existente (Paso 3).

¿Listo para este desafío?

```
1 hat_list = [1, 2, 3, 4, 5] # Esta es una lista existente de números impares
2
3 # Paso 1: escribe una línea de código que solicite al usuario
4 hat_list[2] = int(input("Ingresa un número entero para reemplazar el centro: "))
5
6 # Paso 2: escribe aquí una línea de código que elimine el último elemento de la lista
7 del hat_list[-1]
8
9 # Paso 3: escribe aquí una línea de código que imprima la longitud de la lista existente
10 print("Longitud de la lista:", len(hat_list))
11
12 print(hat_list)
```

Console >_

Ingresa un número entero para reemplazar el centro: 1
Longitud de la lista: 4
[1, 2, 1, 4]

PrevNext

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Python INSTITUTE

« 3.4.1.13 LABORATORIO: Lo básico de las listas - Los Beatles »

MODULE (68%)

SECTION (93%)

LABORATORIO

Tiempo Estimado

10-15 minutos

Nivel de Dificultad

Fácil

Objetivos

Familiarizar al estudiante con:

- Crear y modificar listas simples.
- Utilizar métodos para modificar listas.

Escenario

Los Beatles fueron uno de los grupos de música más populares de la década de 1960 y la banda más vendida en la historia. Algunas personas los consideran el acto más influyente de la era del rock. De hecho, se incluyeron en la compilación de la revista *Time* de las 100 personas más influyentes del siglo XX.

a banda sufrió muchos cambios de formación, que culminaron en 1962 con la formación de John Lennon, Paul McCartney, George Harrison y Richard Starkey (mejor conocido como Ringo Starr).

Escribe un programa que refleje estos cambios y le permita practicar con el concepto de listas. Tu tarea es:

- Paso 1: Crea una lista vacía llamada `beatles`.

```
1 # Paso 1: Crea una lista vacía llamada beatles.
2 beatles = []
3
4 # Paso 2: Emplea el método append() para agregar los siguientes miembros de la banda.
5 beatles.append("John Lennon")
6 beatles.append("Paul McCartney")
7 beatles.append("George Harrison")
8
9 # Paso 3: Emplea el bucle for y el append() para pedirle al usuario que agregue los siguientes miembros de la banda a beatles.
10 for i in range(2):
11     new_member = input("Ingresa el nombre del nuevo miembro: ")
12     beatles.append(new_member)
13
14 # Paso 4: Usa la instrucción del para eliminar a Stu Sutcliffe de la lista.
15 del beatles[-2:]
16
17 # Paso 5: Usa el método insert() para agregar a Ringo Starr al principio de la lista.
18 beatles.insert(0, "Ringo Starr")
19
20 print("Los Fab:", beatles)
```

Console >_

Prev

Next

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome

Python INSTITUTE

« 3.6.1.9 LABORATORIO: Operando con listas - conceptos básicos »

MODULE (87%)

SECTION (90%)

LABORATORIO

Tiempo Estimado

10-15 minutos

Nivel de Dificultad

Fácil

Objetivos

Familiarizar al estudiante con:

- Indexación de listas.
- Utilizar operadores `in` y `not in`.

Escenario

Imagina una lista: no muy larga ni muy complicada, solo una lista simple que contiene algunos números enteros. Algunos de estos números pueden estar repetidos, y esta es la clave. No queremos ninguna repetición. Queremos que sean eliminados.

Tu tarea es escribir un programa que elimine todas las repeticiones de números de la lista. El objetivo es tener una lista en la que todos los números aparezcan no más de una vez.

Nota: Asume que la lista original está ya dentro del código, no tienes que ingresarla desde el teclado. Por supuesto, puedes mejorar el código y agregar una parte que pueda llevar a cabo una conversación con el usuario y obtener todos los datos.

Sugerencia: Te recomendamos que crees una nueva lista como área de trabajo temporal. No necesitas actualizar la lista actual.

```
1 my_list = [1, 2, 4, 4, 1, 4, 2, 6, 2, 9]
2
3 new_list = []
4
5 for number in my_list:
6     if number not in new_list:
7         new_list.append(number)
8
9 my_list = new_list
10
11 print("La lista con elementos únicos:")
12 print(my_list)
```

Console >_

La lista con elementos únicos:
[1, 2, 4, 6, 9]

Prev

Next

Jairo Haziel

Para volver fácilmente a los sitios que visitaste, sincroniza tu historial y tus pestañas con 1225100376jhre@gmail.com

Activar

Contraseñas y autocompletar

Administrar tu Cuenta de Google

Personalizar el perfil

Tu cuenta en Chrome

Salir de Chrome

Otros perfiles de Chrome

Jairo Haziel

Agregar perfil de Chrome

Abrir el perfil de invitado

Administrar perfiles de Chrome